

Multi-Tier Computing

Supplementary Exam Notes

New Content Only — Points not covered in previous notes

Contents

1	Unit I: Introduction — Driving Forces & Open Systems	2
1.1	Fundamental Change in Technology	2
1.2	Driving Forces in the 1990s (Detailed)	2
1.2.1	Key Organizational Drivers	2
1.3	Aspiring to a Single System Image (SSI)	3
1.4	Client/Server Definitions (Detailed)	3
1.5	Modern Client/Server Architecture	4
1.6	Existing Environment: Mainframe-Centric Client/Server	4
1.6.1	Limitations of Dumb Terminals (Earlier Mainframe Model)	4
1.6.2	Advantages of Workstation GUI over Dumb Terminals	4
1.6.3	Electronic Data Interchange (EDI)	4
1.6.4	Intelligent Character Recognition (ICR)	5
2	Open System Standards for Client/Server Computing	5
2.1	Existing Standards (OS and Networks)	5
2.2	Why Open Systems?	5
2.3	Standards Setting Organizations: Factors for Success	5
3	Clients, Servers, and Operating Systems	5
3.1	Detailed Anatomy of a Server Program	5
3.2	What Does a Server Need From an OS? (Detailed)	6
3.2.1	Base Services	6
3.2.2	Extended Services (Detailed)	6
3.3	Server Scalability (Detailed)	7
3.3.1	Asymmetric Multiprocessing	7
3.3.2	Symmetric Multiprocessing (SMP)	7
3.3.3	Clusters	7
4	Client Anatomy (Detailed)	8
4.1	Non-GUI Clients	8
4.2	GUI Clients	8
4.3	Object-Oriented User Interface (OOUI) Clients	8
4.3.1	OOUI Desktop Features	8
4.4	Application Features: GUI vs. OOUI	9
4.5	Compound Documents: OOUIs on Steroids	9
4.6	Shippable Places	9
4.7	What Does a Client Need From the Server?	10
5	Client Requirements: UI Design	10

5.1	UI Design Standards	10
6	Three-Tier Computing: Client/Server Building Blocks	10
6.1	Introduction	10
6.2	Client/Server: A One-Size-Fits-All Model	11
6.2.1	Client/Server for Tiny Shops	11
6.2.2	Client/Server for Small Shops and Departments	11
6.2.3	Client/Server for Intergalactic Enterprises	11
6.2.4	Client/Server for a Post-Scarcity World	12
7	Middleware Building Blocks	12
7.1	Client Building Block	12
7.2	Server Building Block	12
7.3	Middleware Building Block	12
7.4	Pipes and Platforms	13
7.4.1	Pipes	13
7.4.2	Platforms	13
7.5	Server-to-Server Middleware	13
8	Transaction Servers: Additional Details	13
8.1	TP Monitor and ORBs (Detailed)	14
8.2	TP-Lite vs. TP-Heavy: The Argument	14
8.3	TP-Lite vs. TP-Heavy: Resources & Cost	14

Unit I: Introduction — Driving Forces & Open Systems

Fundamental Change in Technology

Computing is in the midst of a fundamental change in both technology and its application. The key constraining factors are:

- The organizational impact of reengineering the business process.
- The costs and time required for system development.

Open Systems enable organizations to buy off-the-shelf solutions to business problems.

- Open systems standards define the format in which data is exchanged, remote systems are accessed, and services are invoked.
- They allow organizations to build reusable class libraries for object-oriented design and development environments.
- They support niche products that interact with the same data (objects).

Client/server computing makes the enterprise available at the desk:

- It provides access to data that previous architectures did not.
- Standards have been defined for client/server computing.
- Powerful enabling technologies with built-in conformance to open systems standards are evolving rapidly (e.g., object-oriented development, relational and object-oriented databases, multimedia, imaging, expert systems, GIS, voice recognition, text management).

Driving Forces in the 1990s (Detailed)

- Opportunities are available to organizations and people who are ready and able to compete in the global market. A competitive global economy will ensure obsolescence to those unwilling to compete.
- Organizations must look for ways to demonstrate value. There is a willingness to rethink existing organizational structures and business practices.
- Organizations are aggressively downsizing even as they try to expand their revenue base, eliminating redundancy and increasing effectiveness.
- Organizations are becoming market-driven while remaining true to their business vision.
- To be competitive globally, organizations in developed economies must employ technology to offset their higher labor costs.
- **Reengineering** the business process to provide information and decision-making support at points of customer contact reduces layers of decision-making management, improves responsiveness, and enhances customer service.
- **Empowerment** means that knowledge and responsibility are available to the employee at the point of customer contact. Client/server computing is the most effective source for tools that empower employees.

Key Organizational Drivers

- Business Process Reengineering
- Globalization — The World as a Market

- Operational Systems — Competition for Investment Dollars
- Market Driven — Flexible to Meet Needs
- Downsized Organizational Structure
- Enterprise Network Management
- Information and Technology Viewed as a Corporate Asset
- Cost Competitive — New Offerings
- Increasing Power and Capacity of Workstations

Aspiring to a Single System Image (SSI)

- In distributed computing, a **Single System Image (SSI)** cluster is a cluster of machines that appears to be one single system.
- The complexity of a heterogeneous computing platform will result in many interfaces at both the logical and physical level.
- There is complexity and risk when attempting to interoperate among technologies from many vendors.
- It is necessary to engage in “*proof of concept*” testing to distinguish the marketing version of products from the delivered version.
- A **Technology Competency Center (TCC)** is a low-cost means of ensuring that the solution is viable. These labs enable rapid installation of the proposed solution into a proven environment.
- The single-system image is best implemented through the client/server model.
- Because the desktop computer is the user’s view into the enterprise, there is no better way to guarantee a single image than to start at the desktop.

Client/Server Definitions (Detailed)

- **Client:** A single-user workstation that provides presentation services and the appropriate computing, connectivity, and database services and interfaces relevant to the business need.
- **Server:** One or more multiuser processors with shared memory providing computing, connectivity, and database services and interfaces relevant to the business need.
- Client/server computing is an environment that satisfies the business need by appropriately allocating the application processing between the client and the server processors.
- The client requests services from the server; the server processes the request and returns the result to the client.
- The communications mechanism is a **message passing interprocess communication (IPC)** that enables (but does not require) distributed placement of the client and server processes.
- **Client/server is a software model of computing, not a hardware definition.**
- This makes client/server a rather generic model fitting what is known as “*cooperative processing*” or “*peer-to-peer*.”
- Because the client/server environment is typically heterogeneous, the hardware platform and operating system of the client and server are not usually the same. In such cases, the communications mechanism may be further extended through a well-defined set of standard **APIs** and **remote procedure calls (RPCs)**.
- Effective client/server computing will be fundamentally **platform-independent**. There is no benefit, yet considerable risk, in exposing the platform to the user.

Modern Client/Server Architecture

- Tools that isolate developers from the specifics of any single platform assist developers in writing transparent, portable applications.
- There are **three essential components** in any application:
 1. **Data Access** — includes the GUI and stored data access.
 2. **Processing** — includes the business logic.
 3. **Interfaces** — link services with other applications.
- The use of **technology layers** provides application development isolation. These layers isolate the characteristics of the technology at each level from the layer above and below.
- This layering is fundamental to the development of applications in the client/server model.
- The rapid rate of change in technologies implies that we must isolate specific technologies from each other.

Existing Environment: Mainframe-Centric Client/Server

- Uses the presentation capabilities of the workstation to **front-end existing applications**.
- Data is displayed or entered through pull-down lists, scrollable fields, check boxes, and buttons.
- User interface is easy to use, and information is presented more clearly.
- Mainframe applications continue to run unmodified, because the existing terminal data stream is processed by the workstation-based communications API.
- Availability of products such as UniKix and IBM's CICS OS/2 can enable the entire mainframe processing application to be moved unmodified to the workstation. This protects the investment in existing applications while improving performance and reducing costs.

Limitations of Dumb Terminals (Earlier Mainframe Model)

- Fixed length fields and fixed length lists.
- Crowded screens with single or limited character fonts.
- Limited or no graphics icons.
- Limited windowing for multiple application display.
- Fixed layout makes it difficult to support display of conditionally derived information.

Advantages of Workstation GUI over Dumb Terminals

- Provides facilities to build the screen dynamically.
- Enables screens to be built with a variable format based conditionally on data values.
- Variable length fields can be scrollable, and lists of fields can have a scrollable number of rows.
- Enables a much larger virtual screen to be used with no additional data communicated between client and server.

Electronic Data Interchange (EDI)

EDI is an example of front-end processing. EDI enables organizations to communicate electronically with their suppliers or customers.

- These systems provide the workstation front end to deal with the EDI link but continue

to work with the existing back-end host system applications.

- Messages are reformatted and responses are handled by the EDI client, but application processing is done by the existing application server.

Intelligent Character Recognition (ICR)

ICR technology can be an extremely effective way to automate the capture of data from a form, without the need to key.

- Current experience shows accuracy rates greater than **99.5%** for typed forms.
- Accuracy rates greater than **98.5%** for handwritten forms.

Open System Standards for Client/Server Computing

Existing Standards (OS and Networks)

- **Unix**
- **POSIX** (IEEE Unix-based)
- **TCP/IP**
- **OSI** (Open Systems Interconnection) — Layered network structure
- **RDA** (Remote Data Access) and **DRDA** (Distributed Relational Database Architecture)

Why Open Systems?

- Conform to a broad set of formal and de facto standards.
- Support for variety of vendors.
- Demands strategic adoption of standards by users and developers.
- Plug and play, easier development.

Standards Setting Organizations: Factors for Success

- Business Motivation
- Internetworking
- Interoperability
- Compatible environments
- Perceived benefits

Clients, Servers, and Operating Systems

Detailed Anatomy of a Server Program

A server program waits for client-initiated requests. Key behavioral characteristics:

- Some servers assign a dedicated session to every client; others create a dynamic pool of reusable sessions; some provide a mix of both.
- Must be able to handle requests without dropping them.
- Executes many requests at the same time (multi-threaded, sometimes additionally multi-tasking).
- Priority-based client request handling.

- Initiates and runs background-task activity.
- **Keeps running:** The server program is typically a mission-critical application.
- **Grows bigger and fatter:** Server programs have an insatiable appetite for memory and processing power. The server environment must be upwardly scalable and modular.

What Does a Server Need From an OS? (Detailed)

Base Services

- **Multitasking:** Server programs exhibit a high level of concurrency. Ideally, a separate task is assigned to each client. Multitasking improves performance, throughput, modularity, and responsiveness.
- **Threads/Coroutines:** Server code runs more efficiently if tasks are allocated to parts of the same program rather than to separate programs. Tasks within the same program are faster to create, faster to context-switch, and have easier access to shared information.
- **Task Preemption.**
- **Task Priority.**
- **Semaphores:** An OS must provide simple synchronization mechanisms for keeping concurrent tasks from bumping into one another when accessing shared resources.
- **Interprocess Communications (IPC):** An OS must provide mechanisms that allow independent processes to exchange and share data.
- Local/Remote Interprocess Communications.
- Threads.
- Intertask Protection.
- Multiuser High-Performance File System.
- Efficient Memory Management.
- Dynamically Linked Run-Time Extensions.

Extended Services (Detailed)

Extended services exploit the distributed potential of networks, provide flexible access to shared information, and make the system easier to manage.

- **Ubiquitous Communications:** OS extensions must provide a rich set of communications protocol stacks.
- **Network Operating System Extensions:** Must provide facilities for extending file and print services over the network; applications should be able to transparently access any remote device.
- **Binary Large Objects (BLOBs):** Images, video, graphics, intelligent documents, and database snapshots are about to test the capabilities of operating systems, databases, and networks.
- **Global Directories and Network Yellow Pages:** OS extensions must provide a way for clients to locate servers and their services.
- **Authentication and Authorization Services.**
- **System Management:** OS extensions must provide an integrated network and system management platform.
- **Network Time:** OS extensions must provide a mechanism for clients and servers to synchronize their clocks.
- **Database and Transaction Services.**

- **Internet Services:** HTTP, SSL, TLS, firewalls, DNS, HTML-based file systems, e-commerce.
- **Object-Oriented Services:** Services are becoming more object-oriented. No current operating system bundles all these extended functions, but they are all aspiring to do so.

Server Scalability (Detailed)

- Clients will always want more services, so scalable servers are frequently an issue.
- **Multiservers (Clusters):** Used in environments that require more processing power than a single server. The client/server model is upwardly scalable — more servers can be added (creating a pool), or the existing server can be traded up.
- Multiservers remove any upward limits to the growth of server power.
- Ordinary servers can also provide this power by working in ensembles using middle-ware.

Asymmetric Multiprocessing

- Imposes hierarchy and a division of labor among processors.
- Only one designated processor, the **master**, can run the operating system at any one time.
- The master controls slave processors dedicated to specific functions such as disk I/O or network I/O.
- A **coprocessor** is an extreme form of codependency: one processor completely controls a slave processor through interlocked special-purpose instructions. The coprocessor has unique special-purpose hardware not identical to the main processor (e.g., graphic coprocessor).

Symmetric Multiprocessing (SMP)

- Treats all processors as equals. Any processor can do the work of any other processor.
- Applications are divided into threads that can run concurrently on any available processor.
- Any processor in the pool can run the OS kernel and execute user-written threads.
- SMP improves performance of the application itself, as well as total throughput of the server system.
- Most importantly, SMP requires new applications that can exploit multithreaded parallelism.
- The few applications that really exploit SMP are SQL DBMSs and TP Monitors.
- Mass-market server operating systems (Unix, NetWare, OS/2, NT) all support SMP.

Clusters

- Make a group of interconnected SMP machines behave like a single system; typically interconnected via a high-speed LAN.
- In 1997, Microsoft brought clustering to NT (**Wolfpack**). The first release provided some level of fault-tolerance via failovers but did not scale.
- Clusters come in two flavors: **shared-disk** and **shared-nothing**.
- Some form of high-speed LAN is used for intercluster communications (simple Ethernet or specialized high-speed redundant bus).
- Clusters provide high-availability because they do not share memory or synchronized caches. Consequently, failures are contained within a single node.

- Some clustering systems use “*I’m alive*” messages to determine the health of each node. If a node dies, messages can be re-routed to an alternate node.
- **Shared-nothing** architectures eliminate bottlenecks found in SMP systems, allowing them to scale to hundreds or even thousands of processors.

Client Anatomy (Detailed)

Client/server applications are **client-centric**. The client side provides the “look-and-feel” for the services a system provides. All client applications have this in common: they request the services of a server.

Based on differences, clients are classified into **three categories**:

Non-GUI Clients

Non-GUI client applications generate server requests with a minimal amount of human interaction. They fall into two sub-categories:

- Non-GUI clients that do not need multitasking from the OS.
- Non-GUI clients that need multitasking from the OS.

GUI Clients

- Simple GUI Clients are applications where occasional requests to the server result from a human interacting with a GUI.
- The simple GUI interface is a good fit for mainstream, OLTP-type business applications.
- They also make good front-end clients to database servers.

Object-Oriented User Interface (OOUI) Clients

- Provides a **highly iconic, object-oriented user interface** that lets you directly manipulate objects on a screen (via drag-and-drop).
- OOUIs are used by information workers doing multiple, variable tasks whose sequence cannot be predicted.
- Examples: executive and decision-support applications, multimedia-based training systems, system management consoles, stockbroker workstations.
- OOUIs have an *insatiable appetite for communications*. OOUI desktop objects need to communicate among themselves and with external servers. Communications are real-time, interactive, and highly concurrent.
- Examples of OOUIs: OS/2 Workplace Shell, NextStep (now Mac OS X), Mac OS, and, to some extent, Windows 98.

OOUI Desktop Features

- Provides a visual desktop metaphor where you can bring together related objects and programs to perform a task.
- The desktop can contain multiple workplaces running concurrently.
- Each workplace may run parallel dialogs (modeless dialogs) over parallel sessions with the server.
- With advanced multimedia applications, you can display images, video, multiobject folders, and voice-annotated mail.

- Information is displayed in the foreground while background tasks are constantly moving information to and from servers. For example, the first page from a multimedia document is displayed while a background task prefetches the rest from the server.
- OOUIs provide folders, workareas, shadows (shortcuts), and associations that allow users to personalize their desktops and manage their objects.
- OOUIs provide a common metaphor for creating, copying, moving, connecting, and deleting any object on the desktop.
- Key feature: **multiple views of objects**.

Application Features: GUI vs. OOUI

Feature	Description
OOUI Integration	Extension of the OS user interface; appears seamlessly integrated.
Drag-and-drop	Transactions triggered by dragging objects (e.g., dragging a guest object to the shredder to delete, or to a fax icon to send confirmation).
Notebook View	The OOUI icon can be opened to reveal a notebook view. The notebook control visually staples together many dialog windows.
Persistent Desktop	The OOUI setup reappears the way the user left it when the machine is turned on again.
Familiarity	Very familiar, especially to kids. Feels like a video game.
Extensibility	Can be seamlessly extended to work with any other OOUI object with very little new code.
GUI (Standard)	Typical Windows or Web interface. The icon represents the application on the desktop. You start the application by clicking on the icon; you are then in menu land. The user is aware that there is a running application.

Compound Documents: OOUIs on Steroids

- Compound document frameworks (e.g., **ActiveX** and **JavaBeans**) are the latest and greatest OOUI technology.
- Every visual object on the screen is a live component.
- Some components are also **containers**, which means they can embed other components.
- You can edit the contents of any component *in-place*, regardless of how deeply embedded it is.
- A component is an independent piece of software that you can separately purchase on the market.
- Components can play together in visual suites that mimic applications.

Shippable Places

- A **place** is a visual ensemble of related components.
- A **shippable place** is a mobile container of components; a place that can be shipped over the Net.
- Today's user interfaces are centered around a primitive place representing a desktop. Shippable places let you interact with *multiple places* that represent collaborative environments based on real-world models.
- A place is a mini virtual world that servers can ship to their clients; a place can be a

dynamic assembly of ever-changing data, video feeds, and other live content.

What Does a Client Need From the Server?

- All client applications need some mechanism to communicate service requests and files to a server.
- All three client categories function best in a robust, multitasking environment.
- It is important to use an OS that can protect programs from clashing and crashing. No client program should cause the system to hang.
- GUI and OOUI clients work best with a thread-like mechanism for handling background requests. Priority-based, preemptive multitasking is required to respond to multimedia devices and to create client applications where multiple dialogs are displayed in parallel.
- One of the most important requirements of a client OS is to host mobile code in a secure fashion. In practice, this means the client OS must provide a **Java VM**.

Client Requirements: UI Design

UI Design Standards

5 User Design Principles:

1. Know the user
2. Simplify repeated tasks
3. Provide feedback to the user
4. Be consistent
5. Test early and often

Other requirements:

- **Interface independence:** for portability.
- Open interface architecture.
- Testing interface.
- Development aids.

Three-Tier Computing: Client/Server Building Blocks

Introduction

Architectures help identify structural elements that may be used as building blocks in the construction of ever-more complex systems.

The key architectural questions are:

- How is the application split between the client and the server?
- What function goes in the client and what function goes in the server?
- Can the client/server model accommodate businesses of all sizes?
- How are nomadic laptop users brought into the client/server fold?
- Can client/server play in the home?

- Where do peer networks fit in this picture?
- Will client/server disappear in a post-scarcity computing world?

Client/Server: A One-Size-Fits-All Model

Three basic building blocks: a client, a server, and the slash (/) that ties the client to the server.

Client/Server for Tiny Shops

- Building-block implementation that runs the client, middleware software, and most of the business services on the same machine.
- Suggested implementation for one-person shops, home offices, and mobile users with well-endowed laptops.
- Easy to run the client and server portions of an application on the same machine.
- Caveat: Need a robust OS that can run both the client and server.

Client/Server for Small Shops and Departments

- Classic Ethernet client/single-server building-block implementation.
- Used in small shops, departments, and branch offices; the predominant form of client/server today.
- Well-suited for LAN-based single server establishments; accounts for around **70%** of today's client/server installations.
- Consists of multiple clients talking to a local server.
- The single-server nature keeps the middleware simple; a client only needs to look into a configuration file to find its server's name.
- Security is implemented at the machine level and kept quite simple.
- The network is usually relatively easy to administer (part-time job).
- No complex interactions between servers, so it is easy to identify failures.
- Data (e.g., a price list) may be downloaded once a day to refresh the local server.
- Departmental servers continue to be popular because they provide a tremendous amount of user autonomy and control.
- With fiber optics and high-speed ATM connections, it will be hard to detect a performance difference between a local departmental server and an enterprise server a continent away.

Client/Server for Intergalactic Enterprises

- Multiserver building-block implementation. Servers present a single-system image to the client but can be spread throughout the enterprise.
- Addresses the needs of establishments with a mix of heterogeneous servers.
- Upwardly scalable: more servers can be added (creating a pool), or the existing server machine can be traded up for the latest generation of superserver machine.
- Servers can be partitioned based on the function they provide, the resource they control, or the database they own.
- Servers can be replicated for fault tolerance or to boost performance.
- Can provide an awesome amount of compute power and flexibility, in many cases rivaling that of mainframes.
- To exploit the full power of multiservers, we need low-cost, high-speed bandwidth and extensive middleware features (including network directory services, network security, RPCs, and network time services).
- Good software architecture for intergalactic enterprise is all about creating system

“ensembles” out of modular building blocks.

- Design servers so they can delegate work to their fellow servers. A complex request may involve a task force of servers working together.
- Preferably, the client should not be made aware of behind-the-scenes collaboration; the server the client first contacts should orchestrate the task force.

Client/Server for a Post-Scarcity World

- Transforms every machine in the world into both a client and a server.
- Personal agents on every machine handle all negotiations with their peer agents anywhere in the universe.
- Every machine is both a client and a full-function server.
- A typical post-scarcity machine: a \$2000 cellular notebook with a top-of-the-line processor, 100 MB of RAM, and 100 GB or more of disk space.
- Such a machine runs all the middleware that vendors will be able to dream of over the next few years.
- Because every machine is a full-function server, it runs at a minimum: a file server, database server, workflow agent, Object Transaction Monitor, and Web server — all connected via an ORB.
- A hundred million machines or more may be running almost all forms of client/server software.

Middleware Building Blocks

Client Building Block

- Runs the client side of the application on an OS that provides a GUI or OOUI and can access distributed services.
- **Thin clients** require a Web browser to download JavaBeans and applets on demand.
- The OS most often passes the buck to the middleware building block and lets it handle the non-local services.
- The client also runs a component of the **Distributed System Management (DSM)** element, ranging from a simple agent on a managed PC to the entire front-end of the DSM application.

Server Building Block

- Runs the server side of the application, typically on top of some shrink-wrapped server software package.
- The five contending server platforms for the next generation of client/server applications are: SQL database servers, TP Monitors, groupware servers, object servers, and the Web.
- The server side depends on the OS to interface with the middleware building block.
- The server also runs a DSM component (could be a simple agent on a managed PC or the entire back-end of the DSM application, e.g., providing a shared object database for storing system management information).

Middleware Building Block

- Runs on both the client and server sides of an application.
- Three categories: **transport stacks**, **network operating systems (NOSs)**, and

service-specific middleware.

- Middleware is the *nervous system* of the client/server infrastructure.
- The middleware also has a DSM software component.

Pipes and Platforms

- The middleware boundaries have become murkier as we move to 3-tier and N-tier client/server.
- The middle-tier application may consist of hundreds or thousands of moving parts. Middleware makes the middle-tier act as a single system (or application).
- In N-tier environments, the middleware must also provide a **platform** for:
 - Running server-side components.
 - Balancing their loads.
 - Managing the integrity of transactions.
 - Maintaining high-availability.
 - Securing the environment.
- It must also provide **pipes** that allow server components to communicate using a variety of metaphors.
- Middleware is still responsible for providing client/server communications pipes.

Pipes

Provide the intercomponent (and interapplication) communication services.

- Examples: RPCs, MOMs, and ORBs.
- Pipes also include wire-level security such as SSL, as well as directory services such as LDAP.

Platforms

Application servers that run the server-side components.

- Can typically be used across multiple operating systems to provide a unified view of the distributed environment.
- Examples: TP Monitors, Object Transaction Monitors, and Web App Servers.
- Note: The platform for the client side is not part of the middleware equation; it is typically either a Web browser or the operating system.

Server-to-Server Middleware

- A server can play both client and server roles.
- Some server-to-server interactions require **specialized server middleware**:
 - A **two-phase commit protocol** may be used to coordinate a transaction that executes on multiple servers.
 - Servers on a mail backbone use special server-to-server middleware for store-and-forward type messaging.
 - Databases and groupware servers use daemons to automatically replicate data.

Transaction Servers: Additional Details

TP Monitor and ORBs (Detailed)

- ORBs bring TP Monitors to the client/server mainstream; they will become the orchestrators of the **Object Web** — the next-generation infrastructure combining Web and object technologies to support Internet, intranet, and extranet applications.
- ORBs provide TP Monitors with a myriad of standard services including: metadata, dynamic invocations, persistence, relationships, events, naming, component factories, versioning, licensing, security, change management, and collections.
- Objects make it easier for TP Monitors to create and manage rich transaction models such as nested transactions and long-lived transactions (workflow).
- TP Monitors will become frameworks for managing objects packaged as transaction-savvy components, e.g., CORBA/EJBs or server-side ActiveXs.

TP-Lite vs. TP-Heavy: The Argument

- **TP-Lite argument:** TP-Lite systems may not solve all the world's problems, but they solve many simple ones. According to Zipf's law: most problems are simple.
- **TP-Heavy argument:** We won't have an OLTP environment manufactured by a single vendor (TP-Lite), but rather an OLTP environment that includes a mosaic of services (TP-Heavy). TP-Heavy is expected to win.

TP-Lite vs. TP-Heavy: Resources & Cost

TP-Lite Example:

- 200 clients, each client has a dedicated pipe/connection to the database.
- No TP Monitor; clients communicate directly with the DB engine.
- Throughput: **20 TPS** (Transactions Per Second).
- Problems: one connection per client is resource-heavy, large number of open pipes creates high overhead, poor scalability, database becomes a bottleneck.

TP-Heavy Example:

- 1000 clients connect to a TP Monitor.
- TP Monitor uses only **24 shared pipes** to the database.
- Throughput: **80 TPS**.
- Database access is controlled and optimized.
- Advantages: connection pooling, reduced database load, better CPU and memory utilization, high scalability, better performance even with more clients.